



CENTRO UNIVERSITÁRIO DE BRASÍLIA

FACULDADE DE TECNOLOGIA E CIÊNCIAS SOCIAIS APLICADAS

CURSO DE CIÊNCIA DA COMPUTAÇÃO

PROJETO INTEGRADOR III

Plano de testes ByteReports

22308138 Áleff Matheus Pinheiro Campos

Brasília

2026

HISTÓRICO DE REVISÕES

Data	Versão	Descrição	Autor(es)
18/05/2026	1.0	Criação do documento.	José Waldo Saraiva Câmara Neto
22/05/2026	1.1	Atualizado os testes internos.	José Waldo Saraiva Câmara Neto
29/05/2026	1.2	Atualização nos testes internos.	José Waldo Saraiva Câmara Neto, Guilherme Miranda Cavalcante, Luiz Felipe Santos de Abreu
04/06/2026	1.3	Atualização nos testes fechados.	José Waldo Saraiva Câmara Neto, Guilherme Miranda Cavalcante
15/06/2026	1.4	Removido menções sobre ferramentas open-source.	José Waldo Saraiva Câmara Neto

SUMÁRIO

HISTÓRICO DE REVISÕES.....	2
SUMÁRIO.....	2
1. Introdução.....	4
1.1 Objetivo do software.....	4
1.2 Objetivo dos testes.....	4
2. Escopo.....	5
3. Abordagem.....	6
4. Missão de avaliação e motivação dos testes.....	7
4.1 Fundamentos.....	7
4.2 Missão de avaliação.....	7
4.3 Motivadores dos testes.....	7
5. Itens de Teste-Alvo.....	8
6. Resumo dos testes planejados.....	9
6.1 Resumo das inclusões dos testes.....	9
6.2 Resumo dos Outros Candidatos a Possível Inclusão.....	9
6.3 Resumo das Exclusões dos Testes.....	9
7. Abordagem dos Testes.....	10
7.1 Tipos e Técnicas de Teste Utilizadas.....	10
7.1.1 Teste de interface do usuário (UI):.....	10
7.1.2 Teste de funcionamento:.....	11
7.1.3 Determinação do perfil de desempenho e carga:.....	12
7.1.4 Teste de tolerância a falhas e de recuperação:.....	13
7.1.5 Teste de configuração e compatibilidade:.....	14
8. Critérios de entrada e de saída.....	15
8.1 Plano de Teste.....	15
8.1.1 Critérios de Entrada de Plano de Teste.....	15
8.1.2 Critérios de Saída de Plano de Teste.....	15
8.1.3 Critérios de Suspensão e de Reinício.....	15
8.2 Critérios de Saída Geral.....	16
8.2.1 Critérios de Entrada de Ciclo de Teste.....	16
8.2.2 Critérios de Saída de Ciclo de Teste.....	17
8.2.3 Término Anormal do Ciclo de Teste.....	18

10. Ambiente de Testes.....	19
10.1 Hardware Básico do Sistema de Testes.....	19
11. Responsabilidades e Papéis da Equipe de Testes.....	20
12. Testes internos.....	22
13. Testes fechados.....	23
14. Testes beta.....	24

1. Introdução

1.1 Objetivo do software

O software a ser testado é o ByteReports, um aplicativo desktop para sistemas operacionais Windows projetado para realizar varreduras locais e diagnósticos de integridade nos principais componentes de hardware (processador, memória RAM, placa de vídeo, armazenamento, bateria e refrigeração).

O sistema extrai métricas via APIs nativas do sistema operacional (como WMI), consolidando os resultados de forma gráfica, simples, didática e local.

1.2 Objetivo dos testes

O propósito do teste realizado é garantir que o ByteReports identifique com precisão os componentes do computador, classifique de forma confiável a gravidade das ocorrências através do sistema de cores (verde, amarelo e vermelho) e gere relatórios e recomendações compreensíveis para usuários leigos, tudo isso de forma ágil, segura (100% local) e estável, sem estourar o limite de consumo de recursos da máquina hospedeira.

2. Escopo

- **Itens que serão testados:**

- As rotinas de varredura automatizada (RF001);
- A exibição e formatação gráfica de dados de hardware (RF002);
- A geração de resumos e relatórios educativos/didáticos (RF003, RF004);
- O tratamento de telemetria ausente e cálculo de expectativa de vida útil (RF005);
- Os gatilhos e instruções passo a passo das recomendações de prevenção (RF006);
- O armazenamento local e histórico de relatórios passados (RF007);
- A validação do gatilho de alerta de menos de 10% de espaço em disco (RF008);
- A exportação de arquivos em formatos PDF e TXT (RF009);
- Os algoritmos de identificação de notebooks e extração de ciclos/desgaste de bateria (RF010);
- O funcionamento dos tooltips flutuantes (RF011);
- As tags visuais de status (RF012);
- Testes de performance de tempo de varredura (limite de 60 segundos), estabilidade, compatibilidade (32/64 bits) e isolamento local de dados (segurança).

- **Itens que não serão testados:**

- Rotinas de execução ou monitoramento em segundo plano (fora do escopo do projeto);
- Mecanismos de alteração, deleção ou modificação direta de arquivos no sistema do usuário;
- Integração com motores de inteligência generativa para relatórios.

3. Abordagem

Os testes serão executados utilizando técnicas de caixa preta com interação direta por meio da Interface Gráfica do Usuário (GUI) desenvolvida em React. Serão utilizados dados válidos e inválidos (simulação de APIs travadas, hardware sem telemetria, falta de espaço em disco e falta de privilégios de administrador) para medir a capacidade de resiliência e as respostas de tratamento de erro do sistema.

- **Critério de iniciação:** Build estável gerado como executável único portátil, repositório do banco de dados local disponível e roteiro de casos de teste estruturado.
- **Critério de suspensão:** Casos em que a varredura trave completamente a aplicação, o executável falhe ao abrir, ou o serviço WMI cause falhas críticas (BSoD) na máquina de teste.
- **Critério de encerramento/aprovação:** Cobertura de 100% dos requisitos funcionais obrigatórios validados com sucesso e ausência de defeitos críticos (Bloqueantes ou de Gravidade Alta).

4. Missão de avaliação e motivação dos testes

4.1 Fundamentos

O ByteReports é desenvolvido por uma equipe de 5 estudantes no âmbito do Projeto Integrador III do curso de Ciência da Computação do CEUB.

Ele preenche a lacuna de softwares complexos de diagnóstico de PC, onde usuários leigos não compreendem o estado real do seu hardware. A arquitetura foi planejada para operar de forma totalmente local no ecossistema Windows utilizando Python no back-end e React no front-end.

4.2 Missão de avaliação

A missão prioritária é verificar uma especificação (requisitos e alegações) e localizar problemas importantes, garantindo que o aplicativo seja amigável e didático o suficiente para o público iniciante, básico e intermediário (que compõe a maior parte do mercado), sem que falhas técnicas maculem a confiabilidade local do sistema.

4.3 Motivadores dos testes

Os testes são motivados por riscos de compatibilidade de arquitetura (32/64 bits), riscos técnicos de falhas de comunicação com o subsistema WMI do Windows, limitações de acessibilidade visual (contraste para daltonismo) e pela necessidade de assegurar a privacidade dos dados de acordo com a premissa de análise 100% local.

5. Itens de teste-alvo

- **Componentes de software produzidos:**
 - Executável Portátil da Aplicação (.exe compilado a partir do Python);
 - Interface Gráfica em React (Telas Inicial, Varredura, Relatório e Recomendações);
 - Rotinas de Persistência de Banco de Dados Local.
- **Componentes de terceiros:**
 - Infraestrutura Windows Management Instrumentation (WMI);
 - Mecanismos de renderização/conversão para PDF/TXT do Windows.
- **Ambientes de hardware alvo:**
 - Computadores Desktop Windows tradicionais;
 - Dispositivos Portáteis (Notebooks para validação da bateria).

6. Resumo dos testes planejados

6.1 Resumo das inclusões dos testes

- Testes de funcionamento da varredura e diagnóstico (RF001 a RF012);
- Testes de interface humano-computador (navegação, caixas de diálogo, tooltips e contraste de cores);
- Testes de performance (validação do tempo total de execução inferior a 60 segundos e uso de memória RAM sob uso do app);
- Testes de configuração e tolerância a falhas (comportamento frente ao WMI desativado e simulação de dados nulos).

6.2 Resumo dos outros candidatos a possível inclusão

- Teste de comportamento sob atualizações automáticas futuras do Windows (investigação sobre quebra de caminhos de chaves do WMI).

6.3 Resumo das exclusões dos testes

- Teste de carga de estresse de rede (excluído pois o aplicativo opera de forma 100% local, sem transmissões para servidores externos).
- Testes de instalação complexa (excluído pois o produto é projetado como executável portátil de clique único, sem instaladores tradicionais).

7. Abordagem dos testes

7.1 Tipos e técnicas de teste utilizadas

7.1.1 Teste de interface do usuário (UI):

Objetivo da técnica:	Experimentar a navegação janela a janela e campo a campo, o comportamento dos elementos gráficos flutuantes (tooltips), as opções de acessibilidade de cores e os modos escuro/claro para observar a conformidade com o planejado.
Técnica:	Percorrer todas as telas do aplicativo (Inicial, Carregamento, Relatório de Diagnóstico e Recomendações). Passar o mouse sobre os termos ("RAM", "CPU", "SSD") para disparar as mensagens flutuantes e alternar entre os modos de cor da interface.
Estratégias:	Validar visualmente e por meio de testes com usuários se o layout apresenta textos breves sem jargões excessivamente técnicos e se o sistema de cores (verde, amarelo, vermelho) ajuda na identificação imediata dos problemas.
Ferramentas necessárias:	Protótipo interativo do Figma para comparação visual e ferramentas de validação de contraste e acessibilidade cromática.
Critérios de êxito:	A técnica suporta o teste de cada tela principal e garante que todos os tooltips sejam exibidos com linguagem simples e didática.
Considerações especiais:	Garantir que o contraste das cores das tags de status ("Saudável", "Atenção", "Crítico") permita a leitura correta por pessoas com daltonismo.

7.1.2 Teste de funcionamento:

Objetivo da técnica:	Experimentar a funcionalidade do aplicativo, incluindo a varredura, detecção de hardware, exibição gráfica e as recomendações práticas a fim de observar e registrar o comportamento-alvo.
Técnica:	Experimentar os fluxos do sistema utilizando dados válidos e inválidos para verificar se os resultados esperados ocorrerão quando forem usados dados válidos (hardware respondendo via telemetria), as mensagens de erro ou de aviso apropriadas serão exibidas quando ocorrerem falhas e cada regra de negócio será aplicada de forma adequada.
Estratégias:	Mapear cliques na interface e validar se a resposta visual condiz com o hardware real da máquina utilizada no teste.
Ferramentas necessárias:	Scripts manuais de validação de requisitos, ferramentas de captura de tela para evidências e o próprio executável do ByteReports.
Critérios de êxito:	A técnica suporta o teste de todos os principais cenários (RF001 a RF012), incluindo listagem clara de nomes comerciais, gráficos de capacidade de disco e geração do relatório didático.
Considerações especiais:	É crítico testar o comportamento do aplicativo rodando em um notebook para validar se as métricas exclusivas de bateria (Cycle Count e desgaste) aparecem corretamente na tela.

7.1.3 Determinação do perfil de desempenho e carga:

Objetivo da técnica:	Experimentar a varredura e a geração do diagnóstico sob condições normais e limites de tempo estabelecidos para registrar o comportamento e os dados de desempenho.
Técnica:	Acionar a varredura completa do sistema cronometrando o tempo decorrido desde o clique inicial até a renderização total da tela de resultados e recomendações. Monitorar o consumo de CPU e Memória RAM do processo do aplicativo durante esse intervalo.
Estratégias:	Realizar a medição consecutiva da varredura para garantir consistência no tempo de resposta e coletar dados de telemetria de uso de hardware gerados pela própria execução do app.
Ferramentas necessárias:	Ferramentas de monitoramento do sistema operando no Windows (Gerenciador de Tarefas do Windows ou semelhantes de terceiros).
Critérios de êxito:	A varredura completa do hardware e a montagem do relatório não ultrapassam o limite estrito de 60 segundos, utilizando poucos recursos do computador de teste.
Considerações especiais:	O teste de desempenho deverá ser executado em máquinas dedicadas e isoladas de outros processos pesados em segundo plano para garantir a exatidão da medição do tempo.

7.1.4 Teste de tolerância a falhas e de recuperação:

Objetivo da técnica:	Simular as condições de falha de infraestrutura de software e experimentar os processos de fallback programados para restaurar ou manter a aplicação estável.
Técnica:	Interromper ou travar manualmente o serviço Windows Management Instrumentation (WMI) do sistema operacional antes de abrir o aplicativo. Forçar cenários em que determinadas peças de hardware não forneçam dados de telemetria compatíveis.
Estratégias:	Verificar se o sistema valida o status do serviço WMI e exibe de forma limpa a mensagem de erro apropriada em vez de travar a aplicação, aplicando o fallback "Informação não detectada" de forma compreensível ao usuário.
Ferramentas necessárias:	Console de Serviços do Windows para desativação de serviços e ferramentas de depuração de código Python.
Critérios de êxito:	O aplicativo permanece estável e funcional mesmo sob a ausência de respostas de telemetria das APIs de hardware.
Considerações especiais:	Garantir que o tratamento da ausência de dados de expectativa de vida útil dos componentes ocorra de forma limpa, sem quebrar o layout da tela final.

7.1.5 Teste de configuração e compatibilidade:

Objetivo da técnica:	Experimentar o objetivo do teste nas configurações de software e de hardware necessárias, a fim de observar o comportamento estável e identificar mudanças de estado.
Técnica:	Executar o executável único do ByteReports em computadores utilizando sistemas operacionais Windows 10 e Windows 11 variando entre arquiteturas de 32 bits e 64 bits. Testar o comportamento em máquinas equipadas apenas com placas de vídeo integradas.
Estratégias:	Confirmar o cumprimento do requisito de portabilidade: o software deve rodar diretamente como um executável único de clique, dispensando completamente a necessidade de o usuário instalar o interpretador Python ou qualquer biblioteca complementar de forma manual.
Ferramentas necessárias:	Ambientes físicos ou máquinas virtuais configuradas com diferentes versões do Windows (Windows 10/11) em 32 e 64 bits.
CrITÉRIOS de êxito:	O aplicativo executa perfeitamente, identifica as especificações corretas e exibe os resultados em todas as variações de arquitetura e placas de vídeo testadas.
Considerações especiais:	Versões legadas ou sem suporte mínimo estabelecido no escopo (como o Windows 8) devem ser testadas para garantir que o bloqueio de escopo impeça a execução com um aviso claro, acionando a ação de contingência mapeada.

8. Critérios de entrada e de saída

8.1 Plano de teste

8.1.1 Critérios de entrada de plano de teste

- O plano de testes deve ser revisado e aprovado pela equipe de desenvolvimento e pela orientação do projeto;
- Os requisitos funcionais (RF001 a RF012) e não funcionais (RNF001 a RNF014) devem estar devidamente mapeados e sem ambiguidades;
- O ambiente físico mínimo de testes (PC Desktop e Notebook Windows) deve estar disponível para uso.

8.1.2 Critérios de saída de plano de teste

- Todos os ciclos de teste planejados (Internos, Fechados e Beta) foram totalmente executados;
- 100% dos casos de teste críticos e mandatórios foram validados;
- Não existem defeitos abertos com severidade "Bloqueante" ou "Crítica" que impeçam o uso seguro do software pelos usuários finais.

8.1.3 Critérios de suspensão e de reinício

- **Suspensão:** Os testes do plano serão suspensos caso o executável único sofra falhas extremas contínuas de inicialização (crash antes da tela inicial) ou se a varredura causar instabilidade crítica no sistema operacional hospedeiro (BSOD).
- **Reinício:** Os testes serão reiniciados após a equipe de desenvolvimento backend/frontend liberar uma nova build corrigida com o respectivo log de alteração no GitHub.

8.2 Critérios de saída geral

8.2.1 Critérios de entrada de ciclo de teste

- **Ciclo 1: Testes internos**

- A sprint de desenvolvimento backend em Python e front-end em React focada nas rotinas de varredura deve estar concluída.
- O código-fonte deve estar integrado e compilado em um executável único portátil (.exe), eliminando a dependência de instalação manual do Python pelo usuário.
- O banco de dados local ou mecanismo de persistência de histórico deve estar pronto para receber operações de escrita e leitura.

- **Ciclo 2: Testes fechados**

- A build do aplicativo deve ter passado com sucesso por todas as verificações do ciclo de testes internos (sem bugs de travamento da varredura).
- O protótipo visual de alta fidelidade e os fluxos no Figma devem estar congelados para servir de gabarito estético.
- A agenda com a professora orientadora Kadidja e convidados acadêmicos deve estar confirmada para a sessão de avaliação presencial ou remota.

- **Ciclo 3: Testes beta**

- O software deve ter sido homologado e aprovado no ciclo de testes fechados, com correções aplicadas para daltônicos e ajustes textuais nos tooltips.
- O website ou repositório oficial do ByteReports deve estar operacional para disponibilizar o download seguro do executável portátil para os stakeholders.
- Os canais de feedback (como formulários ou e-mail de suporte técnico) devem estar criados e prontos para registrar a experiência de uso.

8.2.2 Critérios de saída de ciclo de teste

- **Ciclo 1: Testes internos**

- Comprovação técnica de que o tempo total de varredura de hardware não ultrapassa os 60 segundos regulamentares.
- Verificação bem-sucedida de que nenhum dado pessoal ou de navegação é transmitido para fora da máquina (análise 100% local confirmada).
- Todos os defeitos de código encontrados foram documentados e indexados no repositório do GitHub.

- **Ciclo 2: Testes fechados**

- Validação visual completa de que as caixas de mensagem flutuantes (tooltips) funcionam adequadamente ao passar o mouse ou tocar em siglas como "CPU", "RAM" e "SSD".
- Confirmação de que a linguagem usada nos relatórios atingiu o objetivo didático/educativo, sem jargões excessivamente técnicos.
- Parecer formal da orientadora validando a maturidade do software para distribuição externa.

- **Ciclo 3: Testes beta**

- Coleta e compilação de feedbacks de usuários com nível de conhecimento técnico iniciante, básico ou intermediário.
- Validação real da regra de negócio de exportação: os usuários conseguem gerar e salvar arquivos PDF/TXT formatados de forma limpa para simular o envio a uma assistência técnica.
- Consolidação das métricas de aceitação e satisfação para encerramento da iteração do projeto integrador.

8.2.3 Término anormal do ciclo de teste

O ciclo de teste atual será abortado prematuramente e a build devolvida para a equipe caso ocorra qualquer uma das seguintes condições:

- Identificação de que o aplicativo está editando ou alterando arquivos nativos do computador do usuário (violação direta do escopo estabelecido).
- Se o algoritmo de identificação falhar repetidamente ao diferenciar um computador desktop de um notebook, quebrando a renderização do layout na tela final.
- Se o mecanismo de fallback falhar e o código gerar uma exceção não tratada (crash completo) ao se deparar com uma API que retorne valores nulos ou vazios.

10. Ambiente de testes

10.1 Hardware básico do sistema de testes

- **PC Desktop:** Processador Intel/AMD, Mínimo de 8GB RAM, Armazenamento SSD (para testes gerais de performance).
- **PC Notebook:** Dispositivo portátil Windows (exclusivo para testes de bateria e nível de desgaste - RF010).

10.2 Elementos de software básicos

- **Sistemas operacionais:** Windows 10 e Windows 11 (Arquiteturas de 32 e 64 bits).
- **Software de suporte:** Ferramenta de inspeção de logs locais do Windows.

11. Responsabilidades da Equipe de Testes

Papel no projeto	Responsabilidades específicas nos testes
Product Owner (PO)	● Garante que os testes validem os critérios de aceitação das Histórias de Usuário. ● Combina a missão de avaliação com os motivadores de negócio. ● Defende os interesses de usabilidade para o usuário leigo. ● Avalia a eficiência do esforço de teste no encerramento das Sprints.
Scrum Master	● Supervisiona o gerenciamento, planejamento e logística dos ciclos de testes. ● Garante o cumprimento dos prazos de testes estipulados no cronograma de sprints. ● Remove impedimentos técnicos que bloqueiem a execução dos testes. ● Apresenta relatórios de gerenciamento de defeitos nas reuniões com os stakeholders.
Desenvolvedor Backend	● Define a abordagem técnica referente à arquitetura de automação e de scripts. ● Implementa e valida as rotinas de tratamento de erros e fallbacks do Python e WMI. ● Executa testes unitários nas classes e subsistemas internos. ● Assegura o gerenciamento, manutenção e consistência dos dados de histórico local.

Desenvolvedor Frontend	• Cria os componentes de interface necessários para suportar os requisitos de testabilidade gráfica. • Identifica e descreve os cenários visuais na interface desenvolvida em React. • Registra e mantém atualizada a matriz de rastreabilidade e este plano de testes no GitHub. • Garante a aderência visual do app ao protótipo do Figma.
Testadores	• Implementam e executam manualmente os roteiros de testes detalhados. • Registram os resultados e analisam as falhas para possibilitar a recuperação posterior do sistema. • Documentam os incidentes e abrem solicitações de mudança ou correção de bugs no GitHub. • Verificam se as mensagens de aviso e tooltips aparecem corretamente.

12. Testes internos

Resumo da execução interna:

- **Período de execução:** 02/05/2026 até 30/05/2026
- **Responsável técnico pela validação:** Guilherme Miranda Cavalcante
- **Link para repositório de evidências:** <https://github.com/ByteReports>

Quadro de Casos de Teste - Internos:

ID Caso	Requisito alvo	Descrição do cenário de teste	Resultado esperado	Status	Observações
TI-001	RF001	Usuário abre o aplicativo e vê quais peças do PC estão instaladas.	As informações aparecem corretamente.	Passou	
TC-002	RF001	Abrir o aplicativo com serviço WMI desativado	Mensagem de erro clara orientando o usuário a verificar o WMI (RNF010)	Falhou	Quando acontece apenas aparece erro, sem especificar que foi por conta do WMI
TC-003	RF002	Verificar exibição dos dados do processador	Nome comercial, uso atual (%) e temperatura exibidos de forma gráfica e legível	Parcial	Valor de temperatura ausente, mas uso aparece corretamente.
TC-004	RF002	Verificar exibição da memória RAM	Capacidade total, espaço livre e uso atual exibidos graficamente	Passou	

TC-005	RF002	Verificar exibição das unidades de armazenamento	Tipo (HD/SSD/NVMe), capacidade total e espaço livre exibidos corretamente	Passou	
TC-006	RF003	Verificar tela de resumo ao final da análise por meio de relatório	Tela de diagnóstico exibida com o estado geral da máquina ao término da varredura	Passou	
TC-007	RF004	Verificar presença de textos explicativos sobre cada componente	Texto breve sem jargões técnicos excessivos contextualiza a função de cada peça	Falhou	Essa função não aparece no protótipo usado nos testes internos
TC-008	RF005	Verificar exibição de vida útil estimada quando dados de telemetria disponíveis	Projeção de vida útil restante exibida para o componente	Falhou	Essa função não aparece no protótipo usado nos testes internos
TC-009	RF005	Verificar comportamento quando componente não fornece dados de telemetria	Campo ausente tratado com mensagem limpa, sem crash ou dado vazio sem explicação	Passou	
TC-010	RF006	Verificar recomendações geradas para um alerta identificado	Cada alerta acompanhado de ação corretiva detalhada, passo a passo	Passou	Ações corretivas aparecem no relatório

TC-011	RF007	Executar duas análises e acessar histórico de relatórios	Ambos os relatórios persistem localmente e são acessíveis para comparação	Falhou	Função ainda não presente no protótipo usado no teste interno
TC-012	RF008	Unidade de armazenamento com menos de 10% de espaço livre	Status do componente muda	Passou	O aplicativo avisa o usuário que seu HD/SSD está cheio
TC-013	RF008	Verificar unidade com espaço livre acima de 10%	Nenhum alerta de espaço é exibido para essa unidade	Passou	
TC-014	RF009	Clicar em 'Exportar PDF' na tela de recomendações	Arquivo PDF gerado contendo todas as peças, diagnósticos e recomendações formatados	Passou	
TC-015	RF009	Exportar relatório em formato TXT	Arquivo TXT gerado com as mesmas informações do relatório, de forma legível	Falhou	Função ainda não presente no protótipo usado no teste interno
TC-016	RF010	Executar análise em um notebook	Relatório inclui Cycle Count e percentual de degradação da bateria	Falhou	
TC-017	RF010	Executar análise em um desktop	Seção de bateria não aparece ou exibe mensagem adequada de inaplicabilidade	Passou	

TC-018	RF011	Passar o mouse sobre a sigla 'CPU' na interface	Tooltip com explicação simples e didática aparece ao hover	Falhou	Função ainda não presente no protótipo usado no teste interno
TC-019	RF011	Passar o mouse sobre as siglas 'RAM' e 'SSD'	Tooltips com linguagem simples e curta aparecem para cada sigla	Falhou	Função ainda não presente no protótipo usado no teste interno
TC-020	RF012	Verificar categorização de componente com status 'Saudável'	Badge verde com label 'Saudável' aplicado ao componente e ao cabeçalho de resumo	Parcial	Os relatórios ainda não mostram peças saudáveis, apenas as com problemas
TC-021	RF012	Verificar categorização de componente com status 'Atenção'	Badge amarelo com label 'Atenção' aplicado ao componente e ao cabeçalho de resumo	Passou	
TC-022	RF012	Verificar categorização de componente com status 'Crítico'	Badge vermelho com label 'Crítico' aplicado ao componente e ao cabeçalho de resumo	Passou	
TC-023	RNF001	Monitorar uso de CPU e RAM durante a varredura	O aplicativo utiliza poucos recursos; nenhum impacto perceptível no desempenho do PC	Parcial	Aplicativo ainda usa mais recursos que o planejado, mas pouco

TC-024	RNF002 RNF011	Medir tempo total da varredura completa	Varredura concluída em até 60 segundos	Passou	
TC-025	RNF005	Comparar dados exibidos com informações do Gerenciador de Dispositivos	Dados do aplicativo são condizentes com os dados reais do sistema operacional	Passou	
TC-026	RNF007	Instalar e executar o aplicativo em Windows 10 de 32 bits	Aplicativo instala, abre e executa a varredura sem erros	Não se aplica	A equipe não tem acesso a uma máquina com essas configs, mas se encontrar será testado
TC-027	RNF007	Instalar e executar o aplicativo em Windows 11 de 64 bits	Aplicativo instala, abre e executa a varredura sem erros	Passou	
TC-028	RNF008	Monitorar tráfego de rede durante execução completa do aplicativo	Nenhuma requisição de rede contendo dados do usuário ou do hardware é enviada	Passou	
TC-029	RNF009	Instalar o aplicativo em máquina sem Python instalado	Executável funciona normalmente sem necessidade de instalar Python ou bibliotecas	Não se aplica	Os testes internos foram feitos em ambiente de produção, esse requisito se aplica aos testes beta

TC-030	RNF012	Visualizar interface com simulador de daltonismo (protanopia e deuteranopia)	Status 'Saudável' e 'Crítico' distinguíveis além das cores (forma, ícone ou label)	Falhou	Baixa prioridade, será desenvolvido para as outras etapas de teste
TC-031	RNF013	Alternar entre modo escuro e modo claro	Interface adapta corretamente todos os elementos visuais sem perda de legibilidade	Falhou	Baixa prioridade, será desenvolvido para as outras etapas de teste
TC-032	RNF014	Executar varredura em máquina com apenas placa de vídeo integrada	Placa integrada é detectada e exibida no relatório sem erros	Falhou	O aplicativo ainda não consegue diferenciar entre os tipos de placa de vídeo

13. Testes fechados

Resumo da execução fechada:

- **Período de execução:** 31/05/2026 até 7/06/2026

Quadro de Casos de Teste:

Usuário desktop com Windows 11 64-bit com conhecimento intermediário.

Componente UI	Cenário avaliado	Critério de aceitação	Parecer do avaliador	Sugestões de melhoria/ observações coletadas
Botão "Sobre"	Abrir o app pela primeira vez e ler a proposta antes de clicar em qualquer coisa	Objetivo do app compreendido sem instruções externas	Aprovado	"Ficou claro logo de cara o que o aplicativo faz."
Tooltips	Passar o mouse sobre as peças e ler as explicações	Tooltip aparece no hover com linguagem simples, sem jargão excessivo	Aprovado	"Funcionou bem para todos. O texto é curto e direto."
Botão "Gerar relatório"	Clicar no botão e observar o feedback imediato da interface	Exibir o relatório e a opção de exportar.	Aprovado	
Relatório - processador	Verificar se os dados da CPU estão corretos comparados ao Gerenciador de Tarefas	Nome comercial e uso condizem com os dados reais do sistema	Aprovado	"Bateu certinho com o que eu via no gerenciador de tarefas."

Relatório - RAM	Verificar se uso de memória está coerente com programas abertos no momento	Capacidade total e uso atual exibidos com precisão	Aprovado	"Mostrou 82% de uso, o que bate com as 13GB que eu tinha."
Relatório - GPU	Verificar se uso de memória de vídeo está coerente com programas abertos no momento	Nome comercial e uso condizem com os dados reais do sistema	Reprovado	"Não apareceu a minha GPU."
Desempenho geral	Monitorar CPU e RAM do sistema durante a varredura via Gerenciador de Tarefas	Varredura concluída em até 60s sem impacto perceptível no desempenho do PC	Aprovado	

Usuário notebook com Windows 11 com conhecimento leigo.

Componente UI	Cenário avaliado	Critério de aceitação	Parecer do avaliador	Sugestões de melhoria/ observações coletadas
Botão "Sobre"	Abrir o app e entender o que ele faz antes de qualquer explicação da equipe	Proposta do app compreendida sem instruções externas	Aprovado	"Li a tela e entendi que era pra ver se o computador tá bem."

Tooltips	Passar o mouse sobre as peças e ler as explicações	Tooltip aparece no hover com linguagem simples, sem jargão excessivo	Aprovado	"Deu pra entender o básico de cada coisa"
Relatório	Visualizar a tela de resumo ao final da análise e entender o status geral	Status geral do sistema identificado sem dificuldade logo ao ver a tela	Aprovado	"Vi o 'Atenção' em amarelo e já soube que tinha algo pra olhar."
Textos explicativos	Ler os textos de contextualização dos componentes e avaliar se são compreensíveis	Textos sem jargões excessivos; linguagem didática e acessível ao leigo	Aprovado	"Os textos eram bem simples. Eu entendi o que era cada coisa."
Recomendações	Ler as recomendações geradas	Instruções passo a passo claras e realizáveis pelo usuário sem suporte técnico	Aprovado	

14. Testes beta

Resumo do Beta:

- **Período de execução:** XX/XX/2026 até XX/XX/2026
- **Número de stakeholders externos impactados:**
- **Métricas de sucesso coletadas:**

Quadro de casos de teste internos:

ID Usuário	Configuração da máquina	O aplicativo rodou de primeira?	O relatório gerado foi fácil de entender?	Erro relatado	Feedback
Beta-01		Sim	1 a 5		
Beta-02		Não	1 a 5		

Análise de aderência à proposta de valor:

Referências

BYTEREPORTS. **ByteReports**. GitHub, 2025. Disponível em:
<https://github.com/ByteReports>. Acesso em: 18 maio 2026.

BYTEREPORTS. **Documentação completa - ByteReports**. Brasília, 2026. Disponível em:
https://docs.google.com/document/d/1Z9-8aboUGTBN36h5rb9_Q6OSa-i7wePT75FTFGv3ANc/edit?usp=sharing. Acesso em: 18 maio 2026.

Secretaria de Coordenação e Governança das Empresas Estatais. **TEMPLATE DE PLANO DE TESTES**. Brasília: Ministério do Planejamento, Desenvolvimento e Gestão, 2017. Disponível em:
<https://www.gov.br/gestao/pt-br/assuntos/estatais/central-de-conteudo/kits-governanca-ti/kit-1/processo-de-desenvolvimento-de-software/template-plano-de-testes.docx>. Acesso em: 18 maio 2026.